CODE

Welcome | preprocess.py | vectorizer.py | similarity.py | suggestions.py | model.py | main.py M

main.py > ...

```python
import streamlit as st
from model import analyze_resume

st.title("📄 Hire Sense")

resume = st.text_area("Paste your Resume")
job_desc = st.text_area("Paste Job Description")

def get_grade(score):
    if score >= 80:
        return "A (Excellent)"
    elif score >= 60:
        return "B (Good)"
    elif score >= 40:
        return "C (Average)"
    else:
        return "D (Needs Improvement)"

if st.button("Analyze"):
    if resume and job_desc:
        score, suggestions = analyze_resume(resume, job_desc)

        percentage = round(score * 100, 2)
        grade = get_grade(percentage)

        # ✅ Final Score
        st.success(f"✅ Resume Score: {percentage}%")
        st.info(f"📊 Grade: {grade}")

        # ✅ Progress bar
        st.progress(int(percentage))

        st.subheader("💡 Suggestions:")
        for s in suggestions:
            st.write("- " + s)
    else:
        st.warning("Please enter both resume and job description")
```

main.py | model.py | preprocess.py | similarity.py | suggestions.py | vectorizer.py

model.py

```python
from preprocess import clean_text
from vectorizer import get_vectors
from similarity import calculate_similarity
from suggestions import give_suggestions

def analyze_resume(resume, job_desc):
    clean_resume = clean_text(resume)
    clean_job = clean_text(job_desc)

    vectors = get_vectors([clean_resume, clean_job])
    score = calculate_similarity(vectors)

    suggestions = give_suggestions(resume, job_desc)

    return score, suggestions
```

main.py | model.py | preprocess.py | similarity.py | suggestions.py | vectorizer.py

preprocess.py

```python
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# ✅ Fix (add this)
nltk.download('punkt')
nltk.download('punkt_tab')   # <-- ye line ADD karo
nltk.download('stopwords')

def clean_text(text):
    words = word_tokenize(text.lower())
    filtered = [w for w in words if w.isalnum() and w not in stopwords.words('english')]
    return " ".join(filtered)
```

main.py | model.py | preprocess.py | similarity.py | suggestions.py | vectorizer.py

similarity.py

```python
from sklearn.metrics.pairwise import cosine_similarity

def calculate_similarity(vectors):
    return cosine_similarity(vectors[0:1], vectors[1:2])[0][0]
```

main.py | model.py | preprocess.py | similarity.py | suggestions.py | vectorizer.py

suggestions.py

```python
def give_suggestions(resume, job_desc):
    suggestions = []

    resume = resume.lower()
    job_words = set(job_desc.lower().split())

    missing = [word for word in job_words if word not in resume]

    for word in list(missing)[:5]:
        suggestions.append(f"Consider adding '{word}'")

    if "project" not in resume:
        suggestions.append("Add project section")

    if "skill" not in resume:
        suggestions.append("Add skills section")

    if len(resume.split()) < 80:
        suggestions.append("Increase resume content")

    return suggestions
```

main.py | model.py | preprocess.py | similarity.py | suggestions.py | vectorizer.py

vectorizer.py

```python
from sklearn.feature_extraction.text import TfidfVectorizer

def get_vectors(texts):
    tfidf = TfidfVectorizer()
    return tfidf.fit_transform(texts)
```

INPUT

Hire Sense

Paste your Resume

TEJAS SHARMA
B.Tech Computer Science (Aspirant)
CGPA: 9.8

Paste Job Description

Looking for a motivated developer candidate to assist in software development, including writing clean code and assisting in debugging. Basic knowledge of Python, Data Structures, and Machine Learning is required.

Analyze

OUTPUT

Hire Sense

Paste your Resume

TEJAS SHARMA
B.Tech Computer Science (Aspirant)
CGPA: 9.8

Paste Job Description

Looking for a motivated developer candidate to assist in software development, including writing clean code and assisting in debugging. Basic knowledge of Python, Data Structures, and Machine Learning is required.

Analyze

✅ Resume Score: 44.38%

📊 Grade: C (Average)

💡 Suggestions:

- Consider adding 'intern**'
- Consider adding 'knowledge'
- Consider adding 'including'